

Software Design Document

Visualization of Collaborative Learning Networks Using igraph

Final Year Project | 2026

Table of Contents

1. Introduction
2. System Overview
3. System Architecture
4. Data Design
5. Modules and Components
6. Visualizations
7. Tools and Technologies
8. System Limitations
9. Future Work and Enhancements
10. Conclusion

1. Introduction

1.1 Project Overview

This project aims to develop a software system for visualizing collaborative learning networks using the igraph library. Collaborative learning networks represent the interactions and connections between students in an educational environment. These networks can reveal patterns of collaboration, communication, and knowledge sharing among learners.

The system will process student interaction data and create interactive visual representations of learning networks. This will help educators, researchers, and administrators understand how students collaborate and identify patterns that affect learning outcomes.

1.2 Project Objectives

- Collect and process student collaboration data from various educational sources
- Build a network analysis system to extract meaningful relationships and patterns
- Create interactive visualizations of collaborative learning networks
- Provide tools to analyze network statistics and metrics
- Generate reports and insights about learning collaboration patterns
- Support different types of network analysis (centrality, clustering, community detection)

1.3 Scope

The system will handle:

- Student interaction data (forum posts, group projects, peer reviews)
- Network creation from raw collaboration data
- Multiple visualization types (node-link diagrams, circular layouts, force-directed layouts)
- Network analysis metrics (degree, betweenness, closeness, eigenvector centrality)
- Community detection and clustering
- Interactive features (zoom, pan, node selection, filtering)

Out of scope:

- Real-time data streaming
- Machine learning-based recommendation systems
- Integration with all possible learning management systems

2. System Architecture

2.1 High-Level Description

The Visualization of Collaborative Learning Networks system is a web-based application that transforms raw student interaction data into interactive network visualizations. The system operates in a pipeline:

1. **Data Input:** Accepts student interaction data from various sources
2. **Data Processing:** Cleans and processes data to create network edges and nodes
3. **Network Creation:** Builds network graphs using igraph
4. **Analysis:** Computes network metrics and identifies communities
5. **Visualization:** Renders interactive visualizations using web technologies
6. **Export:** Allows users to save visualizations and analysis results

2.2 Key Features

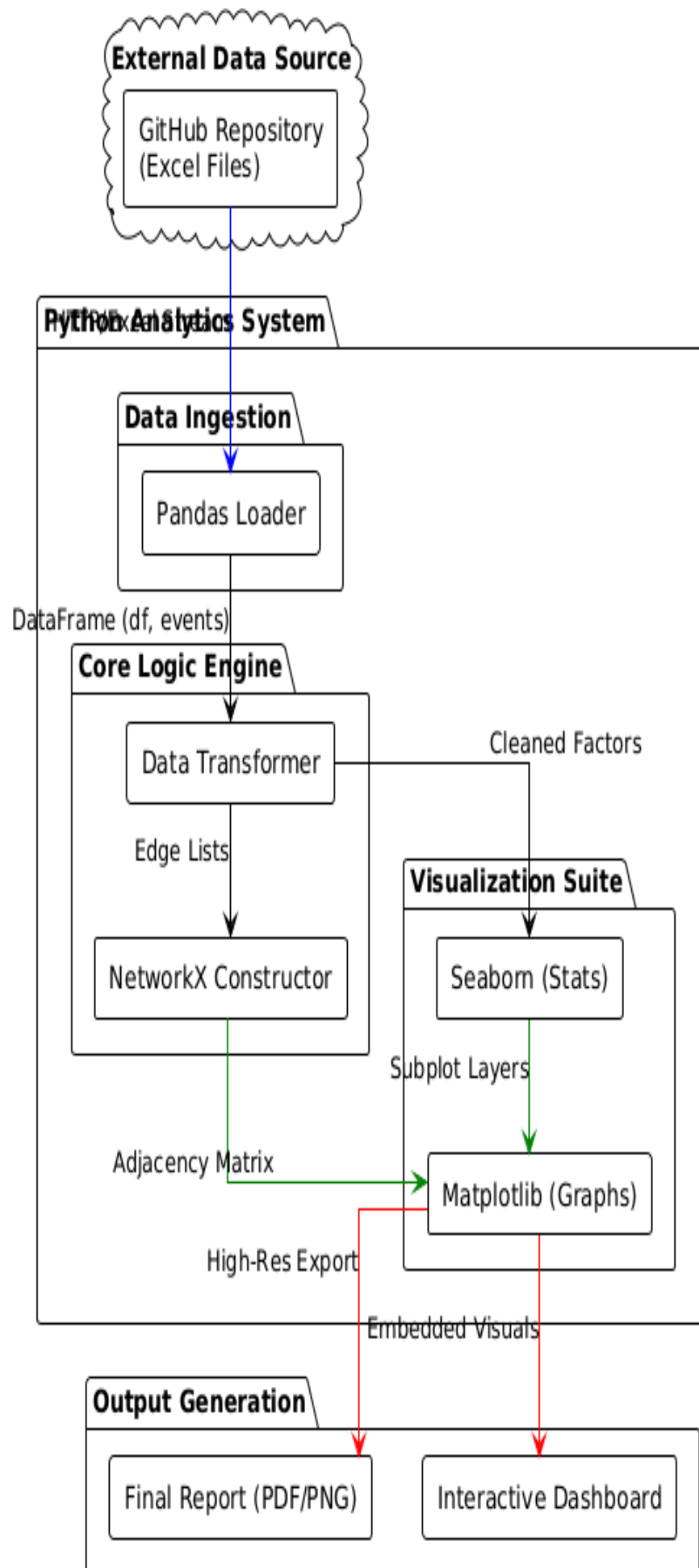
- **Interactive Network Visualizations:** Zoom, pan, and hover over nodes to explore the network
- **Multiple Layout Options:** Circular, hierarchical, force-directed, and spring-embedded layouts
- **Network Metrics:** Display degree, centrality, clustering coefficients, and community structure
- **Filtering and Search:** Find specific students and filter networks by interaction type
- **Customization:** Adjust colors, sizes, labels, and visual properties
- **Export Functionality:** Save visualizations as images or data in various formats
- **Analysis Reports:** Generate summary statistics and insights about the network

2.3 Users and Use Cases

Primary Users:

- **Educators:** Want to understand student collaboration patterns and identify isolated students
- **Researchers:** Analyze learning networks for academic studies
- **Administrators:** Monitor overall collaboration trends in their institution
- **Students:** View their position and connections in the learning network

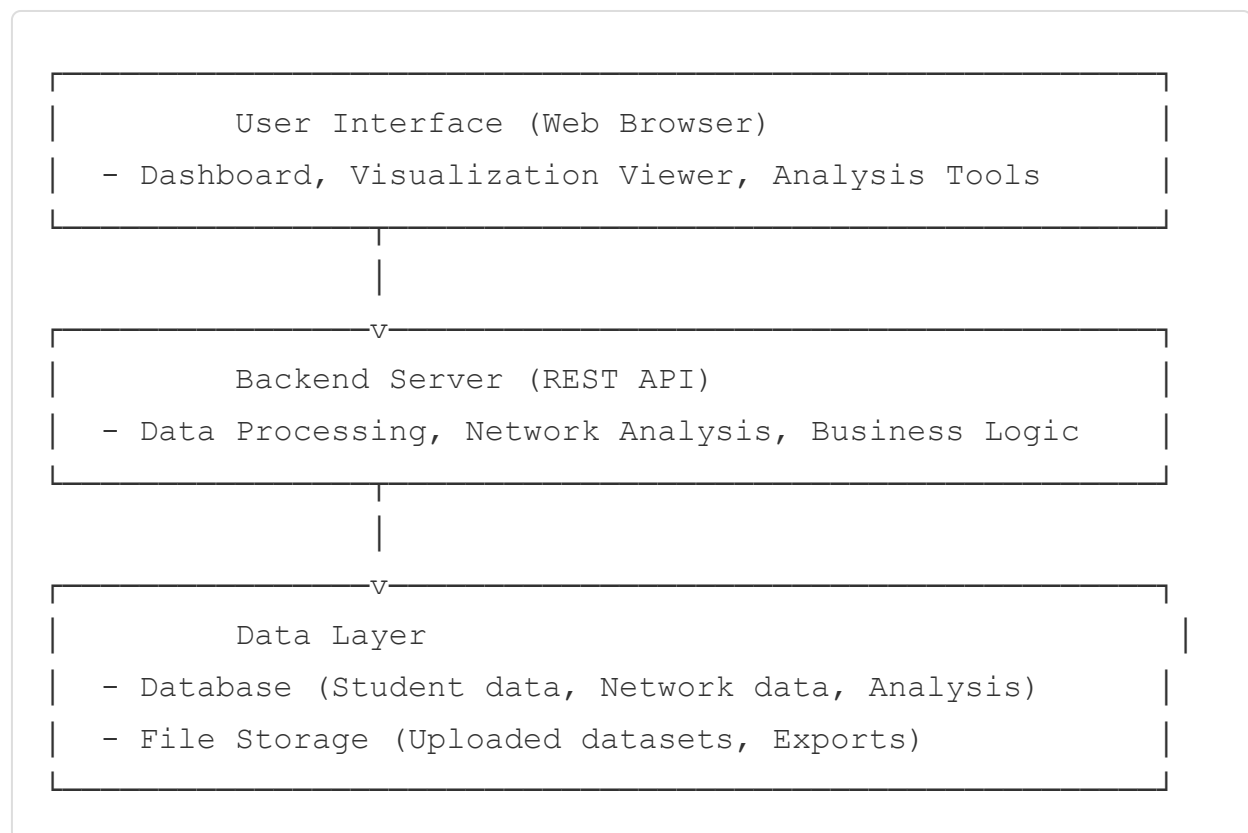
System Architecture: Visualization of Collaborative Learning Networks



Use Cases:

- A teacher uses the system to identify students who need help based on low collaboration
- A researcher analyzes multiple classes to compare learning network structures
- An administrator generates quarterly reports on collaboration trends
- A student checks how well-connected they are to peer groups

3. System overview

Overview Diagram**3.2 Layered Architecture****Presentation Layer:**

- Web-based user interface built with HTML, CSS, and JavaScript
- Interactive visualization using D3.js or similar visualization library
- Responsive design for desktop and tablet devices

Business Logic Layer:

- Data validation and cleaning
- Network construction from raw data
- Network analysis computations
- Community detection algorithms
- Export and reporting functionality

Data Access Layer:

- Database queries and management
- File I/O operations
- Caching mechanisms for performance

External Libraries:

- **igraph:** Core network analysis library
- **Visualization libraries:** D3.js, vis.js, or similar
- **Data processing:** pandas, NumPy (if using Python backend)

3.3 Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, JavaScript, React/Vue.js
Backend	Python (Flask/Django) or Node.js/Express
Visualization	D3.js or Cytoscape.js
Network Analysis	igraph library
Database	PostgreSQL or MongoDB
Data Processing	pandas, NumPy
Server	Apache/Nginx

4. Data Design

4.1 Data Model

Input Data Format

Field	Type	Description
student_id	String	Unique identifier for student
interaction_type	String	Type of interaction (post, comment, group, review)
target_student_id	String	ID of student being interacted with
timestamp	DateTime	When the interaction occurred
course_id	String	Which course the interaction occurred in
interaction_weight	Float	Strength of interaction (0-1)

Network Data Structures

Nodes:

- Student ID
- Student name
- Degree (number of connections)
- Centrality measures
- Community membership
- Achievement group

Edges:

- Source student ID
- Target student ID
- Interaction count

- Interaction types
- Weight/strength of connection
- Date range

4.2 Database Schema

Students Table:

- student_id (PK)
- student_name
- email
- course_id (FK)
- enrollment_date
- achievement_group

Interactions Table:

- interaction_id (PK)
- student_id_from (FK)
- student_id_to (FK)
- interaction_type
- timestamp
- course_id (FK)
- weight

NetworkAnalysis Table:

- analysis_id (PK)
- course_id (FK)
- analysis_date
- total_nodes
- total_edges
- average_degree
- clustering_coefficient
- density
- modularity_score

4.3 Data Flow

Raw interaction data is uploaded or imported

2. Data validation checks for completeness and consistency
3. Duplicates and invalid records are removed
4. Data is aggregated into edges and nodes
5. Network graph is constructed using igraph
6. Analysis metrics are computed and stored
7. Visualization data is prepared for frontend

5. Modules and Components

5.1 Core Modules

5.1.1 Data Import Module

Purpose: Import student interaction data from various sources

Responsibilities:

- Read CSV, Excel, or JSON files
- Validate data format and integrity
- Handle missing or incomplete data
- Support batch uploads
- Log import operations

Functions:

- `import_csv_file(filepath)` - Import from CSV
- `import_excel_file(filepath)` - Import from Excel
- `validate_data_format(data)` - Check data integrity
- `clean_data(data)` - Remove duplicates and handle missing values

5.1.2 Network Construction Module

Purpose: Build network graphs from interaction data

Responsibilities:

- Create nodes from student list
- Create edges from interactions
- Calculate edge weights
- Handle directed vs. undirected graphs
- Support weighted networks

Functions:

- `create_network_graph(nodes, edges)` - Construct igraph object
- `add_node_attributes(graph, attributes)` - Add student properties
- `add_edge_attributes(graph, attributes)` - Add interaction details
- `calculate_weights(interactions)` - Compute edge weights

5.1.3 Network Analysis Module

Purpose: Compute network metrics and properties

Responsibilities:

- Calculate centrality measures
- Detect communities and clusters
- Compute network statistics
- Identify influential students
- Analyze network structure

Functions:

- `calculate_degree_centrality(graph)` - Node degrees
- `calculate_betweenness_centrality(graph)` - Node importance
- `calculate_clustering_coefficient(graph)` - Local clustering
- `detect_communities(graph)` - Community detection using Louvain
- `get_network_statistics(graph)` - Density, diameter, etc.

5.1.4 Visualization Module

Purpose: Generate interactive network visualizations

Responsibilities:

- Layout algorithms (force-directed, circular, hierarchical)
- Render network diagrams
- Apply visual styling (colors, sizes, labels)
- Handle interactivity
- Support multiple output formats

Functions:

- `calculate_layout(graph, layout_type)` - Compute node positions
- `render_visualization(graph, layout)` - Create visual representation
- `apply_styling(visualization, style_config)` - Apply colors and sizes
- `export_visualization(visualization, format)` - Save to image/SVG

5.1.5 Reporting Module

Purpose: Generate analysis reports and summaries

Responsibilities:

- Create statistical summaries
- Generate visualizations of metrics
- Produce PDF/HTML reports
- Support custom report templates

Functions:

- `generate_summary_report(graph)` - Create overview report
- `generate_detailed_analysis(graph)` - Comprehensive metrics
- `export_to_pdf(report)` - Save as PDF
- `export_to_html(report)` - Create interactive HTML report

5.2 API Endpoints

```
GET /api/courses - Get list of courses
POST /api/upload - Upload interaction data
GET /api/networks/{course_id} - Get network data
POST /api/analyze/{course_id} - Run analysis
GET /api/analytics/{course_id} - Get analysis results
POST /api/visualize - Generate visualization
GET /api/export/{format} - Export results
```

6. Visualizations

6.1 Network Visualization Types

6.1.1 Node-Link Diagram

A traditional network graph where:

- **Nodes** represent students (circles)
- **Edges** represent interactions (lines/arrows)
- **Node size** indicates degree or centrality
- **Node color** indicates community or achievement group
- **Edge thickness** indicates interaction strength

Best for: General network exploration and structure understanding

6.1.2 Force-Directed Layout

Uses physics simulation to create aesthetically pleasing layouts:

- Similar to node-link but with better spacing
- Nodes repel each other (like magnets)
- Edges pull connected nodes together
- More interactive and easier to explore
- Good for discovering clusters

Best for: Interactive exploration and cluster visualization

6.1.3 Circular Layout

Arranges nodes in a circle with edges inside:

- Reduces edge crossings
- Good for dense networks
- Edge bundling can show flow patterns
- Hierarchical arrangement possible

Best for: Dense networks and edge pattern analysis

6.1.4 Hierarchical Layout

Organizes nodes by community or achievement:

- Students grouped by community
- Communities arranged hierarchically
- Clear visualization of group structure
- Good for presentation

Best for: Showing community structure and hierarchies

6.2 Summary Visualizations

6.2.1 Degree Distribution

Bar or histogram chart showing:

- Distribution of student connectedness
- How many students have 1, 2, 3... connections
- Identifies hubs and isolated students

6.2.2 Centrality Metrics

Comparison charts showing:

- Which students are most central
- Different types of centrality (degree, betweenness, closeness)
- Rankings and scores

6.2.3 Community Statistics

Visualizations showing:

- Number and sizes of communities
- Inter-community connectivity
- Community composition (high/low achievers)

7. Tools and Technologies

7.1 Core Libraries

igraph

- **Purpose:** Network analysis and graph algorithms
- **Version:** Latest stable (1.x)
- **Why:** Fast, efficient, comprehensive network analysis
- **Key functions:** Community detection, Centrality calculations, Layout algorithms, Path finding

Visualization Libraries

D3.js (if using JavaScript):

- Interactive visualizations
- Force-directed layouts
- DOM manipulation
- Data binding

Cytoscape.js (alternative):

- Specialized for biological networks
- Good community support
- Built-in layouts and algorithms

vis.js (alternative):

- Easy to use network visualization
- Interactive features included

- Good performance

7.2 Backend Framework

Python + Flask:

- Lightweight and flexible
- Good igraph bindings (python-igraph)
- Easy to learn and deploy
- Excellent data processing libraries

Python + Django:

- More features built-in
- Better for larger applications
- ORM for database management

Node.js + Express:

- JavaScript throughout stack
- npm ecosystem
- Good real-time capabilities

7.3 Database Options

PostgreSQL:

- Robust and reliable
- Supports complex queries
- Good for structured data
- Free and open source

MongoDB:

- Flexible schema
- Good for semi-structured data
- Scalable

- JSON-like documents

7.4 Development Tools

Tool	Purpose
Git	Version control
Docker	Containerization and deployment
Jest/pytest	Unit testing
Postman	API testing
VS Code	Code editor
Chrome DevTools	Frontend debugging

8. System Limitations

8.1 Technical Limitations

1. **Network Size:** System performance may degrade with networks > 5000 nodes. Optimization needed for very large networks.
2. **Real-time Processing:** Current implementation processes data in batches. Real-time updates not supported.
3. **Algorithm Complexity:** Some algorithms (e.g., community detection) are computationally expensive. Large networks may require significant processing time.
4. **Visualization Complexity:** Rendering very dense networks can cause UI lag. Edge bundling and filtering help but have limits.
5. **Browser Compatibility:** Advanced visualization features require modern browsers with WebGL support.

8.2 Functional Limitations

1. **Data Sources:** Limited to interaction data formats specified. Custom integrations with all LMS platforms not supported.
2. **Analysis Scope:** Current implementation focuses on structural analysis. Temporal analysis and dynamic networks have limited support.
3. **Multi-university Comparison:** System designed for single institution data. Comparing networks across institutions has limitations due to privacy.
4. **Language Support:** Currently English-only. Internationalization not implemented.
5. **Accessibility:** Not fully WCAG compliant. Improvements needed for visually impaired users.

8.3 User Limitations

1. **Data Privacy:** System requires careful handling of student data. Compliance with FERPA/GDPR needed.
2. **Data Quality:** Results depend heavily on quality of input data. Garbage in, garbage out principle applies.
3. **Interpretation:** Network visualizations can be misinterpreted. User training and guidance needed.
4. **Storage:** Limited to stored data. Real-time or streaming data not supported.

9. Future Work and Enhancements

9.1 Phase 2 Features

1. **Temporal Network Analysis**
 - Animate network evolution over time
 - Show interaction trends
 - Identify emerging clusters
2. **Advanced Community Detection**
 - Overlapping communities
 - Hierarchical clustering
 - Time-aware community detection

3. **Predictive Analytics**

- Predict student performance based on network position
- Identify at-risk students early
- Recommend connections for learning improvement

4. **Multi-layer Networks**

- Support different interaction types as separate layers
- Analyze relationships between layers
- Cross-layer community detection

9.2 Technical Improvements

1. **Performance Optimization**

- GPU acceleration for layout algorithms
- Distributed processing for large networks
- Caching and indexing improvements

2. **Machine Learning Integration**

- Network embedding techniques
- Anomaly detection in collaboration
- Pattern recognition for learning outcomes

3. **Mobile Support**

- Responsive mobile app
- Touch-friendly interactions
- Offline visualization

4. **API Expansion**

- More granular endpoints
- Webhook support
- Bulk operations

9.3 Integration Possibilities

1. **LMS Integration**

- Direct connections to Moodle, Canvas, Blackboard

- Automatic data syncing
- Real-time updates

2. **Learning Analytics Platforms**

- Integration with TLA tools
- Shared data formats
- Combined insights

3. **Educational Research Tools**

- Export to R/Python for further analysis
- Integration with statistical packages
- Academic publication support

9.4 Enhancements for Users

1. **Enhanced Interactivity**

- Drill-down analytics
- Custom filtering options
- Real-time collaboration metrics

2. **Better Reports**

- Customizable templates
- Scheduled report generation
- Email distribution

3. **Recommendations**

- Suggest connections for students
- Recommend study groups
- Identify collaboration opportunities

10. Conclusion

The Visualization of Collaborative Learning Networks using igraph project provides a comprehensive solution for understanding and analyzing student collaboration

patterns. By combining network analysis with interactive visualizations, the system enables educators and researchers to gain valuable insights into learning dynamics.

Key Achievements

- Robust system for processing and analyzing student interaction data
- Interactive visualizations that reveal network structure and patterns
- Comprehensive analysis metrics for understanding collaboration
- User-friendly interface for non-technical educators
- Extensible architecture for future enhancements

Benefits

- **For Educators:** Better understand student interactions and collaboration patterns
- **For Researchers:** Rich data for learning analytics research
- **For Administrators:** Monitor collaboration trends across institution
- **For Students:** Understand their position in learning community

Implementation Success Criteria

1. System successfully processes datasets with 100-5000 students
2. Visualizations render within 5 seconds for typical networks
3. Users can identify isolated students and communities effectively
4. Reports provide actionable insights for educators
5. System maintains data privacy and security standards

Final Remarks

This Software Design Document provides a complete blueprint for developing the Visualization of Collaborative Learning Networks system. The modular architecture ensures flexibility and scalability, while the comprehensive feature set addresses the diverse needs of educational institutions. With proper implementation and testing, this system will become a valuable tool for improving collaborative learning experiences and outcomes.

Software Design Document | Visualization of Collaborative Learning Networks Using igraph

Final Year Project | 2026

© All Rights Reserved